

OpsPilot

Incident Response Copilot

AIOps · LangGraph · Hybrid RAG · Qwen2.5:7B · H100 GPU

Complete Technical Reference & Interview Preparation Guide

Version 4.0 | June 2026 | Balanced Edition

Field	Value
Author	Adarsh Mishra
Organization	Nuvo AI
Core LLM	Qwen2.5:7B via Ollama (Q4_K_M quantisation)
GPU	NVIDIA H100 80 GB — private Docker container

Field	Value
Repository	github.com/adarshmishra121/OpsPilot
Pipeline Status	All 6 DVC stages complete — production ready
Inference Speed	~800 tokens/sec on GPU vs ~1.5 tokens/sec on CPU

Table of Contents

1. Project Overview & Motivation	3
2. System Architecture — The Big Picture	4
3. The HDFS Dataset — Our Training Data	6
4. Drain3 Log Parser — From Raw Logs to Templates	7
5. Feature Engineering — From Templates to ML Vectors	11
6. Anomaly Detection with IsolationForest	13
7. MLflow — Experiment Tracking	16
8. DVC — Reproducible ML Pipeline Management	18
9. RAG System — How We Find Relevant Runbooks	21
10. The LLM Layer — Qwen2.5:7B via Ollama	27
11. LangGraph — The 5-Node Agent State Machine	30
12. Database & Storage Architecture	36
13. FastAPI Backend — API Design & Routing	38
14. Observability — Prometheus, Grafana & Logging	41
15. CI/CD Pipeline — Automated Testing & Deployment	44
16. Streamlit UI — The Engineer's Console	46
17. End-to-End: What Happens When a New Log Arrives	47
18. Interview Q&A; — Deep Dives, Debugging & Motivation	49

1. Project Overview & Motivation

1.1 The Problem We Are Solving

Modern software systems running on Kubernetes generate thousands of log lines and metric signals every minute. When something goes wrong — a node runs out of disk space, a pod crash-loops, an etcd leader election fails — an alert fires and an on-call engineer is paged, often at 3 AM. That engineer must: understand the raw logs, find the right runbook, compose an actionable remediation plan, and verify the fix. Each step can take 10–30 minutes manually. Over a year, this adds up to hundreds of engineering hours lost to repetitive triage work.

OpsPilot automates this workflow. Given an incident — an alert title, a service name, and log lines — it produces a structured, evidence-grounded response in under 5 seconds on GPU. The response includes a natural-language summary, specific remediation actions each citing a real retrieved runbook section, verification commands, a fallback plan, and a draft postmortem. Nothing in the response is invented — every recommended action is provably linked to a retrieved document.

1.2 Why Not Just a Single LLM Call?

Three reasons. **Hallucination:** if you ask any LLM 'how do I fix a Kubernetes node with high disk usage?' it gives plausible-sounding but potentially wrong `kubectl` commands. Our groundedness validator prevents this — if the LLM invents a `doc_id`, that action is silently dropped before it reaches the engineer. **Cost and latency:** running Qwen2.5:7B locally via Ollama costs nothing per request. **Privacy:** production logs contain sensitive hostnames and error messages; sending them to an external API raises compliance concerns.

1.3 OpsPilot Is NOT Fine-Tuning

OpsPilot does **not** fine-tune the large language model. Qwen2.5:7B is used as-is via Ollama. What we train is the **IsolationForest anomaly detector** and the **FAISS+BM25 retrieval index**. The LLM is called only at inference time as a reasoning engine. Training IsolationForest on our log features takes 0.22 seconds.

1.4 Key Metrics

Metric	Value	What It Tells Us
RAG MRR	0.59	Right runbook appears at rank ~1.7 on average
RAG Recall@6	0.58	58% of queries find the gold runbook in top 6
Anomaly training time	0.22 s	Fast enough to retrain daily
P95 API latency (GPU)	~1,000 ms	Full pipeline under 1.5 s on H100
GPU inference speed	~800 tok/s	Near-real-time response generation
CPU inference speed	~1.5 tok/s	Unusable — 18+ min per request

Metric	Value	What It Tells Us
Training data	500,000 HDFS lines	Diverse normal and failure patterns
Runbook corpus	200+ Kubernetes runbooks	Covers all common alert types
DVC pipeline stages	6	download→parse→features→train→index→eval

2. System Architecture — The Big Picture

2.1 Two Distinct Pipelines

OpsPilot is cleanly split into two runtimes. The **offline training pipeline** runs once (or whenever data changes) and produces three artefacts: a trained IsolationForest model file, a FAISS vector index, and a vocabulary file. The **online inference pipeline** runs on every API request and consumes those artefacts. This split is deliberate — training can be slow (minutes) and resource-intensive, while inference must be fast (seconds). You can retrain at 2 AM on fresh log data and deploy the new model without touching the serving infrastructure.

2.2 Offline Pipeline

Raw HDFS log lines flow through Drain3 template mining (converting each line to a cluster ID), through windowed feature engineering (grouping into 500-line windows and building count vectors), and into IsolationForest training. In parallel, 200+ Kubernetes runbook Markdown files are chunked, embedded with SentenceTransformer, and stored in a FAISS index. DVC orchestrates all 6 stages and tracks checksums so only changed stages rerun.

2.3 Online Inference Pipeline

When an incident arrives via HTTP POST, it enters the LangGraph agent — a five-node directed graph. Each node is a pure function that reads from a typed AgentState dictionary and writes back only what it modified. The nodes run in sequence: parse → anomaly → retrieve → draft → validate. The final validated response is serialised by Pydantic and returned as JSON.

2.4 Component Selection Rationale

Component	Technology	Why This Choice
Log parsing	Drain3	Streaming, $O(\log n)/\text{line}$, no labelling needed, stable cluster IDs
Anomaly detection	IsolationForest	Unsupervised, fast, explainable, no labelled anomalies needed
Vector search	FAISS IndexFlatIP	Exact search — fast at 600 chunks, no approximation errors
Keyword search	BM25Okapi	Catches exact alert names semantic search misses
Embeddings	all-MiniLM-L6-v2	22M params, 384-dim, strong quality, disk-cached
LLM	Qwen2.5:7B via Ollama	Best JSON schema compliance among 7B models; free, local
Agent framework	LangGraph	Typed state, independently testable nodes, explicit graph edges
Safety	Groundedness validator	Every action must cite a real retrieved document
Pipeline mgmt	DVC	MD5-based caching, params.yaml for all hyperparameters
Experiment tracking	MLflow	Logs params+metrics+artifacts per training run

Component	Technology	Why This Choice
API framework	FastAPI + Pydantic	Auto validation, auto OpenAPI docs, Prometheus middleware

3. The HDFS Dataset — Our Training Data

3.1 What Is the HDFS Dataset?

The HDFS (Hadoop Distributed File System) log dataset was collected from a 203-node Hadoop cluster running on Amazon EC2 over ~38 hours. The cluster performed standard block read/write operations. Researchers from CUHK injected anomalies and labelled each block access sequence as normal or anomalous, making it the most widely used benchmark in log-based anomaly detection research. The full dataset has ~11.1 million log lines; we cap at 500,000 via `params.yaml` for training feasibility.

3.2 Log Line Format and Why It Matters

Each HDFS log line has six fields: date, time, thread ID, severity level, Java component name, and a free-text message body. The message body contains highly variable content — 128-bit block IDs, IP addresses, port numbers. These variables make direct text modelling impossible: two lines saying the same thing about different blocks look completely different as raw strings. Drain3 solves this by abstracting variables away.

```
# Raw HDFS log line format
081109 203518 143 INFO dfs.DataNode$DataXceiver Receiving block blk_-1608999687 ...
081109 203519 35 ERROR dfs.DataNode$DataXceiver Exception in receiveBlock for blk_-7503 ...
081109 203520 36 WARN dfs.FSdataset Volume /mnt/hadoop/dfs/data failed with ...

# After Drain3 masking + parsing:
blk_-1608999687 → <HEX> (hex mask applied first)
10.251.123.197 → <NUM>.<NUM>.<NUM>.<NUM> (NUM mask)
Both lines share cluster_id = 7, template = "Receiving block <HEX> from /<NUM>.<NUM>.<NUM>.<NUM>"
```

3.3 Why HDFS?

Three reasons: it is real production data from a real distributed system (not synthetic); it has been studied extensively in academic literature giving us benchmark comparisons; and it is large enough to train a meaningful anomaly model but small enough to process quickly. Training IsolationForest on 500k HDFS lines takes 0.22 seconds. We use HDFS to train the model but OpsPilot scores any log lines at inference — Kubernetes logs, application logs, systemd journal. Log template frequency patterns are structurally similar across distributed systems.

4. Drain3 Log Parser — From Raw Logs to Templates

4.1 The Core Problem: Log Variability

Log lines mix static text (the semantic content — what happened) with dynamic values (identifiers — which specific thing it happened to). For machine learning we care about the semantic content and want to abstract away dynamic values. 'Receiving block blk_-1608999 from 10.1.2.3' and 'Receiving block blk_-7503216 from 10.4.5.6' mean exactly the same thing operationally but look completely different as raw strings. Without template extraction, a bag-of-words model treats these as different events and fails to learn they are the same log type.

Drain3 extracts the template 'Receiving block <HEX> from <NUM>.<NUM>.<NUM>.<NUM>' and assigns both lines to the same cluster. The downstream feature engineering can then count how many times this template appeared in a window — a meaningful signal.

4.2 What Is Drain3?

Drain3 implements the Drain algorithm (Data-driven Recursive Abstraction for Ids), published by Pinjia He et al. at ICSE 2017. It is a streaming log parser — it processes one log line at a time, maintains a prefix tree internally, and updates it with each new line. The 'Drain3' variant adds state checkpointing, configurable masking, and production-grade reliability. Every log line is assigned a stable `cluster_id` and a `template_mined` string.

4.3 The Drain Algorithm — How It Works

Drain maintains a parse tree with three node types. The root is the single entry point. Depth-1 nodes split by log line token count — a line with 8 tokens goes to the '8' branch. Depth 2–4 nodes further split on the first significant non-numeric tokens. Leaf nodes hold clusters: a template string and a count of matching lines.

When a new line arrives, Drain walks the tree by length, then by first tokens, reaching candidate clusters. It computes a similarity score: the fraction of token positions that match exactly (neither being a wildcard). If the best score exceeds `sim_th` (0.4 in our config), the line joins that cluster and the template updates — any position that disagrees becomes a wildcard. Below the threshold, a new cluster is created. Masking rules (NUM, HEX) run before the tree lookup.

```
# Drain3 usage pattern (from parse_logs.py)
from drain3 import TemplateMiner
from drain3.template_miner_config import TemplateMinerConfig

config = TemplateMinerConfig()
config.load("drain3.ini")                # sim_th=0.4, depth=4, masking NUM+HEX
miner = TemplateMiner(config=config)      # loads state from checkpoint if exists

results = []
with open("HDFS.log") as f:
    for i, raw_line in enumerate(f):
        if i >= MAX_LINES:
            break
        result = miner.add_log_message(raw_line.strip())
```

```

        results.append({
            "line": raw_line.strip(),
            "cluster_id": result["cluster_id"],
            "template": result["template_mined"],
        })

# Save to Parquet for downstream stages
import pandas as pd
pd.DataFrame(results).to_parquet("data/processed/parsed_logs.parquet", index=False)

```

4.4 drain3.ini — Our Configuration

```

[DRAIN]
sim_th = 0.4          # similarity threshold – lower merges more aggressively
depth = 4             # tokens used for tree branching
max_children = 100    # safety cap on tree branching factor
max_clusters = 100000 # ceiling on distinct templates in memory

[MASKING]
masking_instructions = [{"regex_pattern": "\b\d+\b", "mask_with": "<NUM>"},
                        {"regex_pattern": "[0-9a-f]{8,}", "mask_with": "<HEX>"}]

[SNAPSHOT]
snapshot_interval_minutes = 10 # checkpoint to disk every 10 min (crash recovery)

```

Parameter	Value	Why This Value
sim_th	0.4	Library default; works well on HDFS. Too high → too many clusters; too low → over-merged templates
depth	4	Enough specificity to separate different log types without an overly bushy tree
NUM mask	\b\d+\b	Prevents block counts and port numbers from fragmenting templates
HEX mask	[0-9a-f]{8,}	Block IDs in HDFS are 128-bit integers in hex; critical for HDFS clustering quality
snapshot	10 min	If parse_logs.py crashes mid-500k-lines, resume from checkpoint rather than restart

4.5 Drain3 at Inference Time — Cluster ID Consistency

Drain3 is used *both* offline (parse stage) and online (LogFeaturizer at inference). The same Drain3 state — same configuration, same tree — must process both. If the Drain3 state were reset at inference, a production log matching cluster_id=284 during training might be assigned cluster_id=391 at inference, making the feature vector position wrong and the model score meaningless. We handle this by saving the Drain3 state via checkpointing and loading it at inference — the same template tree that was built during training is used at serving time.

5. Feature Engineering — From Templates to ML Vectors

5.1 Why Feature Engineering?

IsolationForest operates on numeric vectors of fixed dimensionality. Drain3 output is a variable-length sequence of cluster IDs. Feature engineering converts this into a fixed-size vector. A single log line in isolation tells you almost nothing — anomalies reveal themselves as *patterns over time*: a burst of ERROR lines of the same type, a new template appearing that was never seen in training, or the complete absence of expected INFO lines.

5.2 Windowed Aggregation

For each window of 500 consecutive log lines (controlled by params.yaml), we count how many times each known template cluster appears. This produces a frequency histogram — a count vector where position i represents how many times `cluster_id=vocab[i]` appeared in this window. The vocabulary is the top 300 most frequent cluster IDs across all training windows.

```
# Feature vector construction (build_features.py pattern)
import pandas as pd, numpy as np, json

df = pd.read_parquet("data/processed/parsed_logs.parquet")
cluster_ids = df["cluster_id"].tolist()

# Build vocabulary: top-N most frequent cluster IDs
from collections import Counter
freq = Counter(cluster_ids)
vocab = [cid for cid, _ in freq.most_common(VOCAB_SIZE)] # VOCAB_SIZE = 300

# Sliding windows of 500 lines
features = []
for start in range(0, len(cluster_ids) - WINDOW_SIZE, WINDOW_SIZE):
    window = cluster_ids[start : start + WINDOW_SIZE]
    counts = Counter(window)
    vec = np.array([counts.get(v, 0) for v in vocab], dtype=np.float32)
    features.append(vec)

X = np.stack(features) # shape: (~1000, 300)
pd.DataFrame(X).to_parquet("data/features/features.parquet")
json.dump(vocab, open("artifacts/anomaly_vocab.json", "w"))
```

5.3 The Vocabulary — Critical Bridge

The vocabulary file (`artifacts/anomaly_vocab.json`) is a JSON array of 300 cluster IDs in a fixed order, e.g. `[7, 12, 284, 91, ...]`. Position 0 always corresponds to `cluster_id=7`, position 1 to `cluster_id=12`, and so on. The same ordered list is used at training time and inference time. If the order differed, the model's learned weights would correspond to the wrong feature positions — a catastrophic silent failure. The vocabulary is saved by the features stage, tracked by DVC, and loaded by both the training script and the LogFeaturizer at inference.

5.4 Why Count Vectors (Not TF-IDF)?

TF-IDF would suppress the signal from very common templates that suddenly disappear (their IDF weight is low). IsolationForest is sensitive to outliers in the raw feature space — an abnormally high count of any feature is an isolation signal. Raw counts preserve both signals: a burst of rare error templates AND the sudden absence of expected common templates. TF-IDF normalisation would reduce detection quality for the second case.

6. Anomaly Detection with IsolationForest

6.1 Why IsolationForest?

Anomaly detection in log data has a fundamental challenge: **we rarely have labelled anomaly examples**. Real production anomalies are rare and diverse — you might see 10 distinct failure modes over a year, each appearing only once or twice. Supervised classifiers require thousands of labelled examples per class. We don't have that. IsolationForest is **unsupervised** — it learns only from normal operation and identifies future observations that deviate significantly from that normal.

Compared to alternatives: DBSCAN requires hard-to-tune epsilon and min_samples without ground truth. Local Outlier Factor (LOF) must store all training points at inference (slow for large datasets). One-Class SVM is expensive to train. IsolationForest is $O(n \log n)$ to train, $O(\log n)$ to score, requires only one hyperparameter (contamination), and is trivially parallelisable — ideal for daily retraining.

6.2 Algorithm Intuition

The core insight: **anomalies are easier to isolate than normal points**. Build N random trees by repeatedly picking a random feature and a random split point within its range until each point is isolated. An anomalous window (unusual template combination) sits far from the dense cluster of normal windows and gets separated in very few splits. A normal window takes many splits because it is surrounded by similar points. The anomaly score is inversely proportional to average path length across all trees — short path → anomalous.

6.3 Training Code

```
# train_anomaly.py – key sections
import mlflow, joblib, numpy as np, pandas as pd
from sklearn.ensemble import IsolationForest

# Load features from DVC stage output
X = pd.read_parquet("data/features/features.parquet").values.astype(np.float32)

# Train IsolationForest
model = IsolationForest(
    n_estimators=150,      # number of isolation trees
    contamination=0.01,   # expected fraction of anomalies in training data
    random_state=42,      # reproducibility
    n_jobs=-1,            # use all CPU cores
)
model.fit(X)

# Score the training set to compute metrics
raw_scores = model.decision_function(X) # positive = normal, negative = anomalous
anomaly_flags = model.predict(X)        # +1 = normal, -1 = anomalous
anomaly_pct = (anomaly_flags == -1).mean()

# Log everything to MLflow
with mlflow.start_run():
    mlflow.log_params({"n_estimators": 150, "contamination": 0.01, "n_samples": len(X)})
    mlflow.log_metrics({"anomaly_pct": anomaly_pct, "mean_score": raw_scores.mean()})
    mlflow.sklearn.log_model(model, "anomaly_model")
```

```
# Save for DVC tracking
joblib.dump(model, "models/anomaly_model.pkl")
```

6.4 Inference — Score Normalisation

sklearn's `decision_function()` returns raw scores where positive = normal, negative = anomalous. We normalise to [0, 1]: **score = max(0, min(1, 0.5 - raw))**. This flips the sign, centres the scale at 0.5, and clamps to [0, 1]. For our test incident (3 error log lines), raw = -0.0204, normalised = 0.5204 — slightly anomalous but not critical. A score of 0.85+ indicates a clearly anomalous window.

```
# anomaly/infer.py – inference pattern
from functools import lru_cache
import joblib, json, numpy as np

@lru_cache(maxsize=1)
def _load_model():
    return joblib.load("models/anomaly_model.pkl")

@lru_cache(maxsize=1)
def _load_featurizer():
    vocab = json.load(open("artifacts/anomaly_vocab.json"))
    return LogFeaturizer(vocab) # wraps Drain3 + count vector logic

def score_logs(log_lines: list[str]) -> dict:
    if not log_lines:
        return {"score": 0.0, "top_templates": [], "details": {}}

    featurizer = _load_featurizer()
    model = _load_model()

    vec, templates = featurizer.transform(log_lines) # (300,) float32
    raw = model.decision_function(vec.reshape(1, -1))[0]
    score = float(max(0.0, min(1.0, 0.5 - raw))) # normalise to [0,1]

    return {"score": score, "top_templates": templates[:5], "raw_score": raw}
```

6.5 Model Persistence and Hot-Reload

The model is saved via joblib (optimised for numpy arrays). At inference time, it is loaded once via `@lru_cache` and kept in memory for the process lifetime — even under 100 concurrent requests, disk I/O happens exactly once. The admin endpoint `POST /admin/reload` clears the `lru_cache`, forcing a fresh load from disk. This allows hot-reloading a new model after a DVC pipeline run without restarting uvicorn.

7. MLflow — Experiment Tracking

7.1 Why Experiment Tracking?

Without tracking, ML development is trial and error with amnesia. You change contamination from 0.01 to 0.05, retrain, check metrics — but two weeks later you cannot remember which value gave better Recall@6 or explain to a teammate why you chose `n_estimators=150`. MLflow records every training run's inputs (parameters) and outputs (metrics, model files) in a queryable database. Every run is reproducible — the exact config that produced any model is permanently recorded.

7.2 What Gets Logged Per Run

Category	Key	Example Value	Why Logged
Parameters	<code>n_estimators</code>	150	Reproduce exact model configuration
Parameters	<code>contamination</code>	0.01	Most sensitive hyperparameter — affects anomaly threshold
Parameters	<code>n_samples</code>	1000	Documents how much training data was used
Parameters	<code>n_features</code>	300	Documents vocabulary size for this run
Metrics	<code>train_time_s</code>	0.22	Tracks training cost — should stay under 1 second
Metrics	<code>mean_score</code>	0.048	Mean raw score — near 0 for normal training data
Metrics	<code>anomaly_pct</code>	0.25	Fraction flagged — should match contamination setting
Artifact	<code>anomaly_model.pkl</code>	binary	Trained model, linked to this exact run for reproducibility

```
# MLflow tracking pattern (train_anomaly.py)
import mlflow
import time

mlflow.set_experiment("opspilot-anomaly")

with mlflow.start_run(run_name=f"iforest-{{contamination}}"):
    # Log all hyperparameters at run start
    mlflow.log_params({
        "n_estimators": n_estimators,
        "contamination": contamination,
        "n_samples": len(X),
        "n_features": X.shape[1],
        "random_state": 42,
    })

    t0 = time.time()
    model.fit(X)
    train_time = time.time() - t0

    # Log metrics after training
    mlflow.log_metrics({
        "train_time_s": round(train_time, 3),
        "mean_score": float(scores.mean()),
        "std_score": float(scores.std()),
```

```
        "anomaly_pct": float(anomaly_pct),
    })

    # Log the trained model as an artifact (linked to this run)
    mlflow.sklearn.log_model(model, artifact_path="anomaly_model")
```

7.3 Integration with DVC

MLflow and DVC serve complementary purposes. DVC tracks the pipeline at the file level — it knows `models/anomaly_model.pkl` was produced by `train_anomaly.py` from `data/features/features.parquet` with the parameters in `params.yaml`. MLflow tracks the experiment at the run level — exact numeric values of every hyperparameter and metric. Together: DVC tells you which data and code produced which model file; MLflow tells you exactly what it was trained with and how well it performed.

8. DVC — Reproducible ML Pipeline Management

8.1 The Problem DVC Solves

Without pipeline management, ML projects quickly become a mess of shell scripts with files named 'features_final_v3_use_this_one.parquet'. You lose track of which model was trained on which data, accidentally overwrite good models, and teammates cannot reproduce results. DVC treats ML pipelines like software builds: each stage has declared inputs (deps), outputs (outs), and parameters (params). DVC computes MD5 checksums and only reruns stages whose inputs changed.

8.2 dvc.yaml — Our Pipeline Definition

```
# dvc.yaml (simplified structure)
stages:
  download:
    cmd: python scripts/data/download_all.py
    outs:
      - data/raw/hdfs/HDFS.log
      - data/raw/runbooks/

  parse:
    cmd: python scripts/features/parse_logs.py
    deps:
      - data/raw/hdfs/HDFS.log
      - scripts/features/parse_logs.py
    params:
      - params.yaml:
          - data.max_lines          # 500,000
    outs:
      - data/processed/parsed_logs.parquet

  features:
    cmd: python scripts/features/build_features.py
    deps:
      - data/processed/parsed_logs.parquet
    params:
      - params.yaml:
          - features.window_size    # 500
          - features.vocab_size     # 300
    outs:
      - data/features/features.parquet
      - artifacts/anomaly_vocab.json

  train:
    cmd: python scripts/train/train_anomaly.py
    deps:
      - data/features/features.parquet
    params:
      - params.yaml:
          - model.n_estimators      # 150
          - model.contamination     # 0.01
    outs:
      - models/anomaly_model.pkl
    metrics:
      - artifacts/train_metrics.json
```

```

index:
  cmd: python scripts/rag/build_index.py
  deps:
    - data/raw/runbooks/
  params:
    - params.yaml:
      - rag.chunk_size          # 512
      - rag.chunk_overlap       # 64
      - rag.embedding_model     # all-MiniLM-L6-v2
  outs:
    - artifacts/vector_index/

eval:
  cmd: python scripts/eval/run_eval.py
  deps:
    - artifacts/vector_index/
    - data/eval/rag_gold.jsonl
  params:
    - params.yaml:
      - rag.top_k                # 6
  metrics:
    - artifacts/eval_metrics.json

```

8.3 params.yaml — Single Source of Truth

```

# params.yaml – all hyperparameters in one place
data:
  max_lines: 500000

features:
  window_size: 500
  vocab_size: 300

model:
  n_estimators: 150
  contamination: 0.01
  random_state: 42

rag:
  chunk_size: 512
  chunk_overlap: 64
  embedding_model: sentence-transformers/all-MiniLM-L6-v2
  top_k: 6
  alpha: 0.6          # FAISS weight in hybrid fusion (BM25 gets 1-alpha=0.4)

```

8.4 Key DVC Commands and What They Do

Command	What It Does
dvc repro	Run full pipeline, skipping up-to-date stages (checks MD5 of all deps)
dvc status	Show which stages are stale and exactly which file changed
dvc metrics show	Print all tracked metrics: MRR, Recall@6, anomaly_pct, train_time_s
dvc params diff	Compare parameter values between two Git commits
dvc dag	Print pipeline dependency graph as ASCII art
dvc push / pull	Sync data and model files with remote storage (S3, GCS, SSH)
dvc gc	Clean up cached files from old runs to reclaim disk space

8.5 How DVC Detects Changes

DVC stores the MD5 checksum and file size for every tracked file in **dvc.lock**. When you run `dvc repro`, DVC recomputes checksums for all dependency files and compares them to `dvc.lock`. If `params.yaml` changes (e.g. `contamination 0.01 → 0.05`), DVC marks the `train` stage and all downstream stages (`eval`) as stale and reruns only them. The `parse` and `features` stages are unchanged and skipped — saving significant compute time.

9. RAG System — How We Find Relevant Runbooks

9.1 What Is RAG?

RAG (Retrieval-Augmented Generation) grounds the LLM's output in real documents. Instead of asking the LLM to answer from memory (hallucination risk), you first retrieve relevant documents from a knowledge base and then generate an answer grounded in those documents. The LLM's role shifts from 'know everything' to 'synthesise from the provided context.' For OpsPilot, the knowledge base is 200+ Kubernetes runbooks — every recommended action cites a specific retrieved chunk.

9.2 The Runbook Corpus

200+ Kubernetes runbooks from the Prometheus Operator community, stored as Markdown files. Each runbook covers one Kubernetes alert with four sections: Meaning, Impact, Diagnosis, Mitigation. Alerts covered include AlertManager failures, etcd quorum problems, Kubelet issues, node conditions (disk, memory, CPU, network), PV errors, and pod scheduling failures. Runbooks are chunked into ≤512-character segments with 64-character overlap. Each chunk gets a unique `doc_id`: 'runbook:AlertName:chunkIndex'.

9.3 Dense Retrieval: FAISS + Sentence Embeddings

Dense retrieval converts both query and document chunks into 384-dimensional vectors. Texts are semantically similar if their vectors are close — regardless of whether they share the same words. The embedding model is **all-MiniLM-L6-v2** (22M params, sentence-transformers library). All vectors are L2-normalised before storage so that FAISS's IndexFlatIP (inner product) equals cosine similarity. Embedding results are cached by SHA-256 hash on disk via diskcache — rebuilding the index after adding a few runbooks reuses all existing embeddings.

```
# rag/index.py – building the FAISS index
from sentence_transformers import SentenceTransformer
import faiss, numpy as np, json

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

texts = [chunk["text"] for chunk in chunks]      # all runbook chunks
embeds = model.encode(texts, normalize_embeddings=True)  # (N, 384), L2-normalised
embeds = embeds.astype(np.float32)

# IndexFlatIP = exact brute-force inner product (= cosine for L2-normalised vecs)
index = faiss.IndexFlatIP(384)
index.add(embeds)

faiss.write_index(index, "artifacts/vector_index/index.faiss")

# Metadata sidecar: doc_id, title, text, source per chunk
with open("artifacts/vector_index/meta.jsonl", "w") as f:
    for chunk in chunks:
        f.write(json.dumps(chunk) + "\n")
```

9.4 Sparse Retrieval: BM25

BM25 (Best Match 25) is a classical keyword-based ranking function — the backbone of search engines for decades. It scores documents by how many query tokens appear in the document, weighted by token rarity (IDF) and document length. The 'Okapi' variant adds two saturation parameters: $k_1=1.5$ (term frequency saturation) and $b=0.75$ (document length normalisation). We use the `rank-bm25` library (BM25Okapi).

BM25 is essential because alert names like 'KubeNodeUnreachable' are specific strings. Dense search might score 'KubeNodeNotReady' higher because 'unreachable' and 'not ready' are semantically similar. BM25 immediately scores the exact-match runbook very high. BM25 and FAISS fail in complementary ways and complement each other.

```
# rag/bm25.py – BM25 index
from rank_bm25 import BM25Okapi

class BM25Index:
    def __init__(self, texts: list[str]):
        tokenised = [t.lower().split() for t in texts]
        self.bm25 = BM25Okapi(tokenised) # k1=1.5, b=0.75 defaults

    def search(self, query: str, top_k: int = 12) -> list[tuple[int, float]]:
        tokens = query.lower().split()
        scores = self.bm25.get_scores(tokens) # scores for all documents
        ranked = sorted(enumerate(scores), key=lambda x: x[1], reverse=True)
        return ranked[:top_k] # (doc_idx, score) pairs
```

9.5 Hybrid Fusion — Combining Both Signals

Both retrieval systems return scores in different ranges (FAISS: cosine $\sim[-1,1]$; BM25: positive reals with no fixed upper bound). To combine fairly, we min-max normalise each result list independently to $[0,1]$, then compute: **fused** = $0.6 \times \text{normalised_FAISS} + 0.4 \times \text{normalised_BM25}$. Alpha=0.6 was chosen by grid search over $[0.3, 0.5, 0.6, 0.7, 0.9]$ on the 12-query gold evaluation set — alpha=0.6 maximised MRR.

```
# rag/retriever.py – hybrid fusion
import numpy as np

class HybridRetriever:
    def __init__(self, faiss_store, docstore, alpha=0.6):
        self.faiss = faiss_store
        self.docs = docstore
        self.alpha = alpha
        # BM25 built at init time from all documents in docstore
        self.bm25 = BM25Index([d["text"] for d in docstore.all()])

    def retrieve(self, query: str, top_k: int = 6) -> list[dict]:
        # Dense search
        f_results = self.faiss.search(query, top_k=12) # [(doc_id, score), ...]
        # Sparse search
        b_results = self.bm25.search(query, top_k=12) # [(idx, score), ...]

        # Min-max normalise each list independently
        def normalise(pairs):
            scores = [s for _, s in pairs]
            lo, hi = min(scores), max(scores)
            if hi == lo: return {i: 0.5 for i, _ in pairs}
            return {i: (s - lo) / (hi - lo) for i, s in pairs}
```

```

f_norm = normalise(f_results)
b_norm = normalise(b_results)

# Fuse scores across union of retrieved doc IDs
all_ids = set(f_norm) | set(b_norm)
fused = {i: self.alpha * f_norm.get(i, 0) + (1 - self.alpha) * b_norm.get(i, 0)
         for i in all_ids}

ranked = sorted(fused, key=fused.get, reverse=True)[:top_k]
return [{"doc_id": i, "score": fused[i], **self.docs.get(i)} for i in ranked]

```

9.6 RAG Evaluation

Evaluated against 12 test queries in `data/eval/rag_gold.jsonl`. Each entry has a query and the expected `doc_id`. We compute **MRR** (Mean Reciprocal Rank): score of 0.59 means the right document appears at rank $1/0.59 \approx 1.7$ on average. **Recall@6**: 0.58 means 58% of queries find the right document in the top 6. Both metrics are tracked by DVC and saved to `eval_metrics.json` after each index rebuild.

10. The LLM Layer — Qwen2.5:7B via Ollama

10.1 Role of the LLM

The LLM does exactly one thing: take all the structured context assembled by earlier pipeline stages — anomaly score, top log templates, retrieved runbook chunks — and synthesise a coherent, structured JSON response. It does not retrieve, does not detect anomalies. It reads the evidence and writes a clear, specific, actionable remediation plan grounded in that evidence. This is the reasoning step a senior SRE performs after gathering all relevant information.

10.2 Why Qwen2.5:7B?

Compared to Llama3:8B on the same prompts, Qwen2.5:7B consistently returns a JSON object matching our schema. Llama3:8B frequently adds introductory prose or appends explanatory text after the closing brace — both break `json.loads()`. Qwen2.5's instruction fine-tuning was specifically optimised for structured output. We use the **Q4_K_M quantisation** — 4 bits per weight, ~4.4 GB VRAM (down from ~14 GB at full precision) with only small quality degradation. Fits comfortably on any modern GPU.

10.3 Ollama HTTP API — How We Call It

```
# agent/tools.py – call_llm() with streaming + retries
import httpx, json, time

OLLAMA_BASE = os.getenv("OLLAMA_HOST", "http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "qwen2.5:7b")

def call_llm(prompt: str, retries: int = 3, retry_delay: float = 30.0) -> str:
    if LLM_PROVIDER == "mock":
        return _mock_response()

    last_exc = None
    for attempt in range(retries):
        try:
            chunks = []
            # Stream tokens to avoid hard timeout on slow inference
            with httpx.stream(
                "POST",
                f"{OLLAMA_BASE}/api/generate",
                json={"model": OLLAMA_MODEL, "prompt": prompt, "stream": True},
                timeout=httpx.Timeout(
                    connect=10.0,
                    read=1800.0, # 30-min safety net for very long responses
                    write=10.0,
                    pool=10.0
                ),
            ) as resp:
                resp.raise_for_status()
                for line in resp.iter_lines():
                    if line:
                        data = json.loads(line)
                        chunks.append(data.get("response", ""))
                        if data.get("done"):
                            break
            return "".join(chunks)
        except Exception as e:
            last_exc = e
            time.sleep(retry_delay)

    if last_exc:
        raise last_exc
```

```

        return "".join(chunks)
    except (httpx.TimeoutException, httpx.HTTPStatusError) as exc:
        last_exc = exc
        if attempt < retries - 1:
            time.sleep(retry_delay)
        raise RuntimeError(f'Ollama call failed after {retries} attempts') from last_exc

```

10.4 The GPU Problem — A Real Debugging Story

Bug: Every Ollama request succeeded (HTTP 200) but timed out after exactly 5 minutes. The exact 5-minute timeout was suspicious — too precise to be a network issue. Ollama has an internal 5-minute queue timeout for requests waiting for a GPU slot. But we did have a GPU. **Root cause:** The shared Ollama container was started with `--runtime=runc` (the default Docker runtime). Without `--runtime=nvidia`, Docker does not mount CUDA libraries into the container filesystem. Ollama silently falls back to CPU. At 1.5 tok/s, a 1600-token prompt takes 18 min — longer than Ollama's 5-min queue timeout. **Fix:** Started a dedicated opspilot-ollama container on port 11436 with `--runtime=nvidia --gpus device=1`. On H100: same request completes in 2.9 seconds at ~800 tok/s.

```

# Starting the dedicated Ollama container with GPU access
docker run -d \
    --name opspilot-ollama \
    --runtime=nvidia \           # CRITICAL: enables CUDA inside container
    --gpus device=1 \           # use GPU 1 (dedicated to OpsPilot)
    -v ollama_data:/root/.ollama \ # reuse already-downloaded model weights
    -p 11436:11434 \           # separate port from shared container (11434)
    ollama/ollama

# In .env:
OLLAMA_HOST=http://localhost:11436
OLLAMA_MODEL=qwen2.5:7b

```

10.5 The Code Fence Bug

Qwen2.5:7B wraps its JSON output in markdown code fences: ````json...````. This is because it was fine-tuned on coding data where structured output is conventionally fenced. `json.loads()` throws `JSONDecodeError` on the backtick characters. The fix strips fences before parsing:

```

# agent/graph.py – draft_node code fence stripping
def _strip_code_fence(text: str) -> str:
    text = text.strip()
    if "```" not in text:
        return text
    # Find first and last fence positions
    start = text.index("```") + 3          # skip opening ```
    if text[start:start+4] == "json":      # skip optional language tag
        start += 4
    end = text.rindex("```")              # position of closing ```
    return text[start:end].strip()

# In draft_node:
raw_response = call_llm(prompt)
clean_json   = _strip_code_fence(raw_response)
parsed       = json.loads(clean_json)      # now safe to parse

```


11. LangGraph — The 5-Node Agent State Machine

11.1 Why LangGraph?

LangGraph represents the agent as an explicit directed graph where nodes are processing steps and edges are transitions. The state is a typed Python dictionary (TypedDict) that flows through the graph — each node reads from it and writes back only the keys it modifies. We chose it over a simple function chain for three reasons: **typed state** (AgentState documents exactly what flows through the system), **testability** (each node is a pure function testable in isolation without mocking the full pipeline), and **extensibility** (adding a conditional branch requires adding one node and one edge, not restructuring calling code).

11.2 AgentState — The Data Contract

```
# agent/graph.py – AgentState TypedDict (the data contract)
from typing import TypedDict, Any

class AgentState(TypedDict):
    incident: dict          # raw incident from API request
    query: str              # retrieval query (set by parse_node)
    log_lines: list[str]    # log lines (set by parse_node)
    anomaly_result: dict     # score, templates (set by anomaly_node)
    retrieved_chunks: list[dict] # top-6 runbook chunks (set by retrieve_node)
    draft_response: dict     # LLM JSON output (set by draft_node)
    final_response: dict     # validated response (set by validate_node)

# Graph construction
from langgraph.graph import StateGraph, END

builder = StateGraph(AgentState)
builder.add_node("parse", parse_node)
builder.add_node("anomaly", anomaly_node)
builder.add_node("retrieve", retrieve_node)
builder.add_node("draft", draft_node)
builder.add_node("validate", validate_node)

builder.set_entry_point("parse")
builder.add_edge("parse", "anomaly")
builder.add_edge("anomaly", "retrieve")
builder.add_edge("retrieve", "draft")
builder.add_edge("draft", "validate")
builder.add_edge("validate", END)

graph = builder.compile()
```

11.3 The Five Nodes — What Each Does

Node 1: parse_node — Query Extraction

Extracts fields from the raw incident dictionary and prepares them for downstream nodes. Constructs the retrieval query by concatenating alert_title and service. This combined query is more specific than the alert title alone — including the service name helps retrieve runbooks that mention both the alert type and similar services. Executes in microseconds — pure string manipulation, no I/O.

```
def parse_node(state: AgentState) -> dict:
    inc = state["incident"]
    query = f"{inc['alert_title']} {inc.get('service', '')}".strip()
    return {
        "query": query,
        "log_lines": inc.get("log_lines", []),
    }
```

Node 2: anomaly_node — IsolationForest Scoring

Calls `score_logs()` with the log lines from state. If no log lines were submitted, returns `score=0.0` immediately. The anomaly score appears in the LLM prompt as quantified context ('anomaly score: 0.48 on 0–1 scale') and in the API response for the Streamlit UI to display. The model and featurizer are both `@lru_cache'd` — the first request loads them from disk; subsequent requests use the in-memory copy.

```
def anomaly_node(state: AgentState) -> dict:
    log_lines = state.get("log_lines", [])
    if not log_lines:
        return {"anomaly_result": {"score": 0.0, "top_templates": [], "details": {}}}
    result = score_logs(log_lines) # uses @lru_cache'd model + featurizer
    return {"anomaly_result": result}
```

Node 3: retrieve_node — Hybrid RAG Search

Calls the `HybridRetriever` (also `@lru_cache'd`) with the query from `parse_node` and requests the top 6 results. The FAISS index is queried for semantic similarity and BM25 for keyword relevance. Results are fused with 60/40 weighting and the top 6 unique chunks are returned with their `doc_ids`. Takes a few milliseconds — FAISS search on 600 vectors is microseconds; BM25 scoring is also very fast.

```
def retrieve_node(state: AgentState) -> dict:
    retriever = _get_retriever() # @lru_cache – loaded once at startup
    chunks = retriever.retrieve(state["query"], top_k=6)
    return {"retrieved_chunks": chunks}
```

Node 4: draft_node — LLM Synthesis

The most complex node. Formats the `SYSTEM_PROMPT` with anomaly score, top templates, retrieved context text, and incident JSON. The retrieved context is formatted as a numbered list where each chunk is prefixed with its `doc_id` in brackets — showing the LLM the exact `doc_id` format makes it more likely to cite correctly. After calling the LLM, strips code fences and parses JSON. If parsing fails, degrades gracefully: raw LLM text placed in summary field, all other fields empty defaults.

```
def draft_node(state: AgentState) -> dict:
    # Format retrieved context with doc_ids prominently shown
    context_text = "\n\n".join(
        f"[{c['doc_id']}] {c['text']}" for c in state["retrieved_chunks"]
    )
    prompt = SYSTEM_PROMPT.format(
        anomaly_score = state["anomaly_result"]["score"],
        top_templates = state["anomaly_result"].get("top_templates", []),
        context        = context_text,
        incident_json  = json.dumps(state["incident"], indent=2),
    )
    raw = call_llm(prompt)
    clean = _strip_code_fence(raw)
    try:
        draft = json.loads(clean)
    except json.JSONDecodeError:
        draft = {"summary": clean, "actions": [], "verification_steps": []},
```

```

        "fallback_plan": [], "postmortem_markdown": ""}
    return {"draft_response": draft}

```

Node 5: validate_node — Groundedness Enforcement

Collects all doc_ids from retrieved_chunks (the known_ids set). Checks every action: at least one of its evidence_doc_ids must appear in known_ids. Actions with no valid evidence are logged via structlog (warning) and silently dropped — they never appear in the final API response. This is code-enforced, not just a prompt instruction. Even if the LLM hallucinates a convincing but non-existent doc_id, that action is removed.

```

def validate_node(state: AgentState) -> dict:
    known_ids = {c["doc_id"] for c in state["retrieved_chunks"]}
    clean_actions = []

    for action in state["draft_response"].get("actions", []):
        refs = set(action.get("evidence_doc_ids", []))
        valid_refs = refs & known_ids      # intersection: only truly retrieved docs

        if not valid_refs:
            structlog.get_logger().warning(
                "action_rejected_no_evidence",
                action=action.get("action", ""),
                refs=list(refs),
            )
            continue      # silently drop – never reaches the API response

        clean_actions.append(**action, "evidence_doc_ids": sorted(valid_refs))

    final = {**state["draft_response"], "actions": clean_actions}
    return {"final_response": final}

```

12. Database & Storage Architecture

12.1 SQLite in Dev, PostgreSQL in Production

OpsPilot uses **SQLModel** as its ORM — built on SQLAlchemy and Pydantic. The same class validates API request/response data and defines the database schema, eliminating duplication. The database URL comes from the `DATABASE_URL` environment variable. In local dev: `'sqlite:///./opspilot.db'`. In Docker Compose production: `'postgresql+psycopg://opspilot:opspilot@postgres:5432/opspilot'`. Application code is identical in both cases — SQLModel abstracts over the backend.

```
# storage/models.py – FeedbackRow table
from sqlmodel import SQLModel, Field, Column
from sqlalchemy import JSON
from datetime import datetime
from typing import Optional

class FeedbackRow(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    incident_id: str = Field(index=True) # fast lookup by incident
    helpful: bool # engineer's verdict
    comment: Optional[str] = None # free-text note
    tags: Optional[list] = Field(default=None, sa_column=Column(JSON)) # flexible categorical labels
    created_at: datetime = Field(default_factory=datetime.utcnow)

# storage/db.py – dependency injection pattern
from sqlmodel import create_engine, Session
from contextlib import contextmanager

engine = create_engine(os.getenv("DATABASE_URL", "sqlite:///./opspilot.db"))

def get_session():
    with Session(engine) as session:
        yield session # FastAPI DI: session committed after route returns
```

12.2 Storage Systems Summary

System	Technology	What's Stored	File Location
Feedback DB	SQLModel + SQLite/PG	Engineer feedback on responses	opspilot.db
FAISS index	faiss binary	384-dim embeddings for ~600 chunks	artifacts/vector_index/index.faiss
Document metadata	JSONL	doc_id, title, text, source per chunk	artifacts/vector_index/meta.jsonl
Anomaly model	joblib pickle	Trained IsolationForest weights	models/anomaly_model.pkl
Vocabulary	JSON	Top-300 cluster IDs in fixed order	artifacts/anomaly_vocab.json
Embedding cache	diskcache (SQLite)	SHA-256 → float32 vector	`.cache/embeddings/`
MLflow tracking	SQLite + filesystem	Params, metrics, artifact paths	mlflow.db + mlruns/

System	Technology	What's Stored	File Location
Parsed logs	Apache Parquet	500k rows: line, cluster_id, template	data/processed/
Feature matrix	Apache Parquet	~1000 × 300 float32 feature vectors	data/features/

13. FastAPI Backend — API Design & Routing

13.1 Why FastAPI?

FastAPI gives us automatic request validation (Pydantic), automatic interactive OpenAPI docs at /docs, and Prometheus middleware support — all with minimal boilerplate. Every request body is validated before the route function is called; invalid requests return a detailed 422 with field-by-field errors automatically.

13.2 The Critical load_dotenv() Bug

Bug: tools.py reads LLM_PROVIDER = os.getenv('LLM_PROVIDER', 'mock') at module level — this runs at import time. The original main.py imported tools.py BEFORE calling load_dotenv(). So os.getenv ran before .env was loaded, always returning the default 'mock'. The .env file had LLM_PROVIDER=ollama but it was never read. Every API call used mock mode. **Fix:** load_dotenv() must be the absolute first call in main.py, before any local import. Ruff E402 is suppressed for this file in pyproject.toml with a comment explaining why.

```
# src/opspilot/api/main.py – correct ordering (load_dotenv FIRST)
from pathlib import Path
from dotenv import load_dotenv

load_dotenv(Path(__file__).parents[3] / ".env") # MUST be before local imports

from fastapi import FastAPI # stdlib/third-party OK after
from opspilot.api.routes.incident import router # local imports after load_dotenv
```

13.3 Endpoints

Endpoint	File	Purpose
GET /health	health.py	Liveness probe. Returns {status, version}. Excluded from Prometheus metrics.
POST /incident/analyze	incident.py	Main endpoint. Invokes compiled LangGraph agent. Returns IncidentAnalysisResponse.
POST /rag/search	rag.py	Exposes hybrid retrieval without LLM. For testing retrieval quality in isolation.
POST /anomaly/score	anomaly.py	Exposes IsolationForest scoring without retrieval. For batch log scoring.
POST /feedback	feedback.py	Persists FeedbackRow to DB via SQLAlchemy session.
POST /admin/reload	admin.py	Clears @lru_cache on model + retriever. Hot-reload after DVC pipeline run.
GET /metrics	auto (instrumentat or)	Prometheus text format metrics for all instrumented routes.

13.4 Pydantic Schemas

```
# api/schemas.py – request and response models
from pydantic import BaseModel
```

```
from typing import Optional

class IncidentRequest(BaseModel):
    incident_id: str
    alert_title: str
    service: Optional[str] = None
    environment: Optional[str] = "production"
    log_lines: list[str] = []

class RemediationAction(BaseModel):
    action: str
    evidence_doc_ids: list[str] # must be non-empty after safety validation

class AnomalyReport(BaseModel):
    score: float # [0.0, 1.0] - 0=normal, 1=highly anomalous
    top_templates: list[str]

class IncidentAnalysisResponse(BaseModel):
    incident_id: str
    summary: str
    anomaly_report: AnomalyReport
    retrieved_context: list[dict]
    actions: list[RemediationAction]
    verification_steps: list[str]
    fallback_plan: list[str]
    postmortem_markdown: str
```

14. Observability — Prometheus, Grafana & Logging

14.1 Why Observability Matters for AIOps

An incident response system that goes down silently is worse than no system — engineers might trust it when it is not functioning. OpsPilot needs observability at three levels: request-level metrics (requests/sec, latency, error rate), system-level metrics (is Ollama running, is GPU saturated), and log-level diagnostics (why did a specific request fail, which actions were dropped by the validator).

14.2 Prometheus — Auto-Instrumentation

The `prometheus-fastapi-instrumentator` library hooks into FastAPI's middleware system. It automatically exposes two metric types: a **counter** (`http_requests_total`) labelled by handler, method, and status code; and a **histogram** (`http_request_duration_seconds`) recording request duration in configurable buckets. Histograms enable percentile computation via `histogram_quantile()` in PromQL — P95, P99 latency over any time window.

```
# observability/metrics.py
from prometheus_fastapi_instrumentator import Instrumentator

def instrument_app(app):
    Instrumentator(
        should_group_status_codes=False,    # track 200 vs 422 vs 500 separately
        excluded_handlers=["/health"],      # don't track liveness probe noise
    ).instrument(app).expose(app)          # adds GET /metrics endpoint

# PromQL queries used in Grafana dashboard:
# P95 latency for /incident/analyze:
#   histogram_quantile(0.95,
#     rate(http_request_duration_seconds_bucket{handler="/incident/analyze"}[5m]))
#
# Requests per second:
#   rate(http_requests_total[1m])
#
# Error rate (non-2xx):
#   rate(http_requests_total{status!~"2.."}[5m])
```

14.3 Prometheus Configuration

```
# prometheus.yml – scrape config for OpsPilot API
global:
    scrape_interval: 10s      # collect metrics every 10 seconds

scrape_configs:
    - job_name: "opspilot-api"
      static_configs:
        - targets: ["host.docker.internal:8000"]  # native API from Docker container
          labels:
            service: "opspilot"

# Started Prometheus with custom config due to port conflicts:
# Port 9090 taken → used 9092
# Added --add-host=host.docker.internal:host-gateway so container can reach native API
docker run -d --name opspilot-prometheus \
```



```
-p 9092:9090 \
--add-host=host.docker.internal:host-gateway \
-v /tmp/opsilot-prometheus.yml:/etc/prometheus/prometheus.yml:ro \
prom/prometheus
```

14.4 Grafana Dashboard

The OpsPilot dashboard was provisioned programmatically via the Grafana HTTP API. This means the dashboard definition is reproducible — no manual clicking required. The dashboard auto-refreshes every 10 seconds and shows 30 minutes of history. Four panels: P50/P95/P99 latency for `/incident/analyze`, requests/sec by status code, error rate (non-2xx), and P95 latency across all endpoints.

Port 3000 was already taken by another service (merlin-frontend). Grafana runs on port 3001 instead. The Prometheus datasource was added via the Grafana API pointing to `http://opsilot-prometheus:9090` (using Docker internal DNS).

14.5 Structured Logging with structlog

structlog produces JSON-formatted log lines where every field is a named key-value pair. Instead of unstructured text `'action rejected: Delete everything'`, we get machine-parseable JSON that can be counted, filtered, and alerted on in any log aggregator (Elasticsearch, Grafana Loki, CloudWatch).

```
# observability/logging.py
import structlog, logging

def configure_logging():
    structlog.configure(
        processors=[
            structlog.contextvars.merge_contextvars,
            structlog.processors.add_log_level,
            structlog.processors.TimeStamper(fmt="iso"),
            structlog.processors.JSONRenderer(), # machine-parseable JSON output
        ],
        wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),
    )

# Usage in validate_node – structured fields, not f-strings:
structlog.get_logger().warning(
    "action_rejected_no_evidence",
    action=action.get("action", ""),
    refs=list(refs),
    known_ids=sorted(known_ids),
)

# Output: {"event":"action_rejected_no_evidence","action":"Delete everything",
#         "refs":["fake:doc:99"],"level":"warning","timestamp":"2026-06-06T13:41:32Z"}
```

15. CI/CD Pipeline — Automated Testing & Deployment

15.1 Why CI/CD for an ML Project?

ML projects need CI/CD more than typical software projects — the regression surface is larger. A code change might break an API endpoint; a parameter change might degrade model quality; a Docker change might make the image unbuildable. Without automated checks, any of these could silently reach production.

15.2 ci.yml — The Main Workflow

```
# .github/workflows/ci.yml (key structure)
on:
  push:    {branches: [main, dev]}
  pull_request: {branches: [main]}

jobs:
  lint:
    steps:
      - run: python -m ruff check .      # style + import ordering
      - run: python -m mypy src/         # type checking

  test:
    needs: lint
    env:
      LLM_PROVIDER: mock                # no GPU needed in CI — tests use mock LLM
      PYTHONPATH: src
    steps:
      - run: pip install -e ".[api,ui]"
      - run: python tests/setup_dummy_test_data.py  # create stub artifacts
      - run: python -m pytest tests/ -v

  dvc-check:
    needs: test
    steps:
      - run: dvc status                  # warns if pipeline is out of date

  deploy:
    needs: [lint, test, dvc-check]
    if: github.ref == 'refs/heads/main'  # only on main branch merges
    steps:
      - uses: appleboy/ssh-action@master
        with:
          host:    ${ secrets.DEPLOY_HOST }
          username: ${ secrets.DEPLOY_USER }
          key:      ${ secrets.DEPLOY_SSH_KEY }
          script: |
            cd ~/work/OpsPilot
            git pull origin main
            pip install -e .
            pkill -f "uvicorn opspilot" || true
            nohup uvicorn opspilot.api.main:app --port 8000 &
```

15.3 latency-report.yml — Daily P95 Gate

```
# .github/workflows/latency-report.yml (key section)
```

```

on:
  schedule: [{cron: "0 8 * * *"}]    # daily at 08:00 UTC
  workflow_dispatch:                 # allow manual trigger

jobs:
  latency-check:
    env: {LLM_PROVIDER: mock}
    steps:
      - run: nohup uvicorn opspilot.api.main:app --port 8000 & sleep 4
      - run: |
          python - <<'EOF'
          import httpx, time, statistics, sys
          PAYLOAD = {"incident_id": "CI-001", "alert_title": "NodeDiskRunningFull",
                     "service": "payment-api", "log_lines": ["ERROR disk at 95%"]}
          latencies = []
          for _ in range(10):
              t0 = time.perf_counter()
              httpx.post("http://localhost:8000/incident/analyze",
                        json=PAYLOAD, timeout=30).raise_for_status()
              latencies.append(time.perf_counter() - t0)
          p95 = sorted(latencies)[int(len(latencies)*0.95)]
          print(f"P95: {p95*1000:.0f}ms")
          if p95 * 1000 > 2000:    # fail if P95 > 2 seconds
              sys.exit(1)
          EOF

```

15.4 Test Strategy

Tests run in mock mode — no GPU, no Ollama, no database. Two test files cover different layers. **test_agent_safety.py** tests the groundedness validator in isolation — passes grounded actions, rejects ungrounded, handles edge cases (empty evidence, partially valid evidence). **test_api_contract.py** tests every API endpoint using FastAPI's TestClient (runs the API in-process without a real HTTP server). The LangGraph agent is mocked at the route level.

16. Streamlit UI — The Engineer's Console

16.1 Design Philosophy

The Streamlit UI is intentionally thin — it is a client over the FastAPI backend. Every validation, every ML computation, every LLM call happens in the API. The UI's only job is to provide a convenient form for submitting incidents and a structured view for reading results. This means the UI can be modified, restarted, or replaced without affecting any backend functionality.

16.2 Key UI Code Patterns

```
# ui/streamlit_app.py – key patterns
```

```
# Sidebar input form
```

```
with st.sidebar:
```

```
    incident_id = st.text_input("Incident ID", value="INC-2026-0001")
    alert_title = st.text_input("Alert Title", value="NodeDiskRunningFull")
    log_lines_raw = st.text_area("Log Lines (one per line)", height=200)
    analyze_btn = st.button("Analyze Incident", type="primary")
```

```
if analyze_btn:
```

```
    log_lines = [line.strip() for line in log_lines_raw.strip().split("\n") if line.strip()]
    payload = {"incident_id": incident_id, "alert_title": alert_title, "log_lines": log_lines}
```

```
    with st.spinner("Running agent: parse → anomaly → retrieve → draft → validate..."):
```

```
        try:
```

```
            resp = httpx.post(f"{API_URL}/incident/analyze", json=payload, timeout=120.0)
            resp.raise_for_status()
            data = resp.json()
```

```
        except httpx.ConnectError:
```

```
            st.error("Cannot connect to API. Is uvicorn running on port 8000?")
            st.stop()
```

```
# Metrics row
```

```
col1, col2, col3 = st.columns(3)
```

```
col1.metric("Anomaly Score", f"{data['anomaly_report']['score']:.2f}")
```

```
col2.metric("Log Lines Analyzed", len(log_lines))
```

```
col3.metric("Runbooks Retrieved", len(data.get("retrieved_context", [])))
```

```
# Results in tabs
```

```
tab1, tab2, tab3, tab4 = st.tabs(["Summary", "Context", "Actions", "Postmortem"])
```

```
with tab1:
```

```
    st.write(data.get("summary", ""))
```

```
with tab3:
```

```
    for i, action in enumerate(data.get("actions", [])):
```

```
        st.markdown(f"***{i+1}.** {action['action']}")
```

```
        st.caption(f"Evidence: {' '.join(action['evidence_doc_ids'])}")
```

17. End-to-End: What Happens When a New Log Arrives

This section traces a single incident request through the entire system — from HTTP request to JSON response. All the components described in previous sections interact here.

The Incident

```
POST /incident/analyze
{
  "incident_id": "INC-2026-0047",
  "alert_title": "NodeDiskRunningFull",
  "service": "payment-api",
  "environment": "production",
  "log_lines": [
    "ERROR disk usage at 95% on /dev/sda1",
    "WARN inode count critical on node-42",
    "ERROR write failed: no space left on device"
  ]
}
```

Step 1: FastAPI Validation (~1 ms)

FastAPI's Pydantic validation runs immediately. All required fields are present and correctly typed — HTTP 422 is not triggered. The request body is deserialised into an IncidentRequest object and passed to the route function, which calls agent.invoke() with the incident dict as the initial state.

Step 2: parse_node (~0.1 ms)

Extracts alert_title='NodeDiskRunningFull' and service='payment-api', constructs query='NodeDiskRunningFull payment-api'. Passes log_lines through. No I/O — pure string manipulation.

Step 3: anomaly_node (~5 ms)

score_logs() runs the three log lines through the cached LogFeaturizer. Drain3 assigns templates: 'ERROR disk usage at % on /dev/sda' → cluster 284; 'WARN inode count critical on node-' → cluster 91; 'ERROR write failed: no space left on device' → cluster 156. The feature vector has counts [0,...,1,1,...,0,1,...,0] at positions for clusters 284, 91, 156 in the vocabulary. IsolationForest.decision_function() returns raw=-0.0204. Normalised score = 0.5204.

Step 4: retrieve_node (~5 ms)

HybridRetriever queries both FAISS and BM25 with 'NodeDiskRunningFull payment-api'. BM25 immediately gives the KubeNodeDiskRunningFull runbook a very high score (exact string match). FAISS adds semantically similar runbooks about disk usage, inode exhaustion, and node filesystem pressure. After fusion with alpha=0.6, the top 6 chunks are returned.

Step 5: draft_node (~2,400 ms — GPU inference dominates)

The SYSTEM_PROMPT is formatted with all context: anomaly score 0.52, top templates, the 6 runbook chunks (each prefixed with its doc_id), and the incident JSON. Prompt is ~1,600 tokens. Qwen2.5:7B on

H100 generates ~400 tokens at 800 tok/s — completes in ~0.5 seconds. Code fences stripped. `json.loads()` succeeds. The draft response has 3 actions, each citing valid `doc_ids`.

Step 6: `validate_node` (~0.5 ms)

Known `doc_ids` set built from retrieved chunks. All 3 actions in the draft cite `doc_ids` that appear in the retrieved set — all 3 pass validation. No actions are dropped in this case.

Step 7: Response Assembly (~1 ms)

The incident route maps the final state to `IncidentAnalysisResponse`. Pydantic serialises to JSON. FastAPI sends HTTP 200. The `prometheus-fastapi-instrumentator` middleware records the total request duration in the histogram and increments the request counter. `structlog` writes a structured JSON log line with `incident_id`, anomaly score, and action count.

Total time on H100 GPU: ~2.5 seconds. Breakdown: Pydantic validation 1ms, parse 0.1ms, anomaly 5ms, retrieve 5ms, LLM draft 2,400ms, validate 0.5ms, serialisation 1ms.

18. Interview Q&A; — Deep Dives, Debugging & Motivation

18.1 Motivation & Vision

■ Q: What motivated you to build OpsPilot?

I was looking at the on-call workflow and realised it has the same structure every time: read logs, find the relevant runbook, write a remediation plan, verify it worked. Each step is mechanical and repetitive. The SRE's real expertise is in edge cases — the incident that doesn't match any runbook, the subtle interaction between two failing components. I wanted to automate the mechanical part so the engineer can focus on the truly hard cases. The key insight was that RAG plus a safety validator gives you LLM synthesis without hallucination — every recommendation is provably linked to a retrieved document. That combination makes the system trustworthy enough to act on at 3 AM.

■ Q: Why not just use GPT-4 and call it done?

Three reasons. Hallucination — GPT-4 will confidently recommend kubectl commands that don't exist. Our groundedness validator prevents this by requiring every action to cite a document that was actually retrieved. Cost — running Qwen2.5:7B locally costs nothing per request. For a team handling 500 incidents per month, that matters. Privacy — production logs contain sensitive information; sending them to an external API raises compliance concerns. Running everything locally keeps sensitive data on-premises.

18.2 Architecture

■ Q: Why did you choose LangGraph over a simple function chain?

LangGraph gives two things a function chain doesn't. Typed state via AgentState TypedDict — every field is named and typed. When I added anomaly_result, my IDE immediately told me which nodes needed to handle it. And independently testable nodes — I can call parse_node({'incident': {...}}) directly in a test without instantiating the full graph. In a function chain, you tend to test as an integrated whole, making it harder to pinpoint failures. The downside is LangGraph adds a dependency and requires understanding its compilation model.

■ Q: Why FAISS + BM25 hybrid instead of just one retrieval method?

Dense and sparse search fail in complementary ways. FAISS captures semantic similarity but can fail on exact keyword matches — 'KubeNodeUnreachable' might embed closer to 'KubeNodeNotReady' because the concepts are similar. BM25 handles this perfectly — it scores the exact-match runbook very high. A pure BM25 system misses synonyms; a pure FAISS system misses exact names. Alpha=0.6 was set by grid search on our 12-query gold set. The hybrid is consistently better than either alone.

■ Q: How does IsolationForest work, and what does a score of 0.48 mean?

Anomalies are rare and different, so they are easier to isolate. Build N random trees by repeatedly picking a random feature and split point until each point is isolated. An anomalous window ends up in its own leaf after very few splits — it sits far from the dense cluster of normal points. A normal window

takes many splits. sklearn returns a raw score where negative = anomalous, positive = normal. I normalise with score = 0.5 - raw, clamped to [0,1]. For our test case, raw = -0.0204, normalised = 0.5204 — slightly anomalous but not critical.

18.3 Debugging Stories

■ Q: What was the hardest bug you encountered?

The 'always mock' bug. I had LLM_PROVIDER=ollama in .env but every response clearly came from the mock LLM. I added print statements and confirmed LLM_PROVIDER was 'mock' inside tools.py — even though .env said 'ollama'. The clue: os.environ.get('LLM_PROVIDER') at the top of main.py returned 'ollama', but the same call inside tools.py returned 'mock'. Python executes module-level code immediately when the module is first imported. main.py called load_dotenv() on line 9 but imported opspilot.agent.tools on line 7 — tools.py had already read LLM_PROVIDER with the default value 'mock' before load_dotenv() ran. The fix: load_dotenv() as the absolute first call in main.py, before any local import.

■ Q: Tell me about the GPU problem.

Every request to the shared Ollama container succeeded with HTTP 200 but timed out after exactly 5 minutes. The exact 5-minute timeout was suspicious — Ollama's internal queue timeout. We had a GPU but the container's docker inspect showed Runtime: runc (the default). Without --runtime=nvidia, Docker does not mount CUDA libraries. Ollama falls back silently to CPU. At 1.5 tok/s on CPU, a 1600-token prompt takes 18 minutes — longer than the 5-minute queue timeout. I couldn't modify the shared container (another project used it). Solution: dedicated opspilot-ollama container on port 11436 with --runtime=nvidia --gpus device=1, sharing the model weights volume read-only. On H100: 2.9 seconds instead of 18 minutes.

■ Q: What was the code fence issue?

Qwen2.5:7B wraps its JSON output in ```json...``` markdown fences. It was fine-tuned on coding data where structured output is conventionally fenced. json.loads() throws JSONDecodeError on the backticks. My first fix used split('```', 2)[-1] — but split with maxsplit=2 gives three parts, and [-1] is the empty string after the closing fence. The correct fix uses index() and rindex() to find the character positions of the first and last fence, then slices between them.

■ Q: What was the str.format() KeyError in the system prompt?

The SYSTEM_PROMPT used str.format() to inject four values. The prompt also included a JSON schema example showing what the output should look like. JSON uses curly braces everywhere. Python's str.format() interprets every { and } in the string as a placeholder boundary. So {"summary": "..."} caused format() to look for a key named "summary" in the kwargs — which doesn't exist. KeyError: "summary". Fix: escape all literal curly braces in the format string as {{ and }} — format() converts {{ to a literal { in the output.

18.4 Scalability & Future Work

■ Q: How would you scale OpsPilot to 100 requests per second?

The bottleneck is the LLM — Ollama processes one request at a time per GPU. At 2.5 seconds per request, one GPU handles ~0.4 RPS. To reach 100 RPS you need a vLLM server instead of Ollama — vLLM implements continuous batching, packing multiple requests into a single GPU forward pass. A single H100 running vLLM can handle 50–100 RPS for a 7B model. The FastAPI layer is horizontally scalable — all state lives in model files and FAISS index (loaded into memory), not in API processes. Multiple API pods behind a load balancer sharing a single vLLM server.

■ Q: What would you improve with more time?

Three things. First, close the feedback loop: FeedbackRow records engineer judgements but we never use them. A weekly reranking step where helpful=True feedback boosts runbooks cited in successful responses would make OpsPilot improve with every resolved incident. Second, richer context: currently only alert_title and log_lines. Adding Prometheus metric values at alert time (CPU, memory, disk I/O) would give the LLM much more context, reducing cases where retrieved runbooks don't match the actual failure mode. Third, multi-hop retrieval: for complex cascading failures, a second retrieval pass using the LLM's initial summary as the query would capture secondary runbooks relevant to root cause rather than just symptom.

■ Q: How do you ensure the system never recommends harmful actions?

Three layers. The SYSTEM_PROMPT instructs: 'If no relevant context was found, say so honestly — do NOT invent solutions.' This primes the model to acknowledge uncertainty. The schema requires evidence_doc_ids for every action, forcing explicit citation. The safety validator is hard enforcement — it is code that runs after the model responds. Any action whose evidence_doc_ids are not in the retrieved set is silently dropped. Even if the model hallucinates a convincing but non-existent runbook reference, that action never appears in the output. Prompt engineering (soft) plus code enforcement (hard) is more robust than either alone.